

Title of Topic: JOINS

Course Code: 23IT201

Course Title: DataBase Management Systems

Academic Year: 2024 - 2025 (Even Sem)

Module 2

JOINS

JOINS

Topics to be Covered

- Joins
 - ✓ Natural Join
 - ✓ Inner Join
 - ✓ Outer Join
 - ❖ Left Outer Join
 - ❖ Right Outer Join
 - ❖ Full Outer Join

Join Operations

Natural Join

- Operates on two relations and produces a relation as the result
- Considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations

E.g., SELECT * FROM Student S NATURAL JOIN Marks M;

Student Table

Roll_No	Name
1	A
2	B
3	C

Marks Table

Roll_No	Marks
2	70
3	50
4	85

Output

Natural Join

Roll_No	Name	Marks
2	B	70
3	C	50

Join: Part 1:



Part 2:



Join Operations

Inner Join

- Selects all rows from both the tables as long as the condition satisfies
- Create the result-set by combining all rows from both the tables where the value of the common field will be same

E.g., `SELECT * FROM student S INNER JOIN Marks M ON S.Roll_No = M.Roll_No;`

Output

Student Table		Marks Table		Inner Join			
Roll_No	Name	Roll_No	Marks	Roll_No	Name	Roll_No	Marks
1	A	2	70	2	B	2	70
2	B	3	50	3	C	3	50
3	C	4	85				

Join Operations

Outer Join

- Three forms of outer join
- ✓ The left outer join (left join) preserves tuples only in the relation named before (to the left of) the left outer join operation
- ✓ The right outer join (right join) preserves tuples only in the relation named after (to the right of) the right outer join operation
- ✓ The full outer join (full join) preserves tuples in both relations

Join Operations

Left Outer Join

- Returns all the rows of the table on the left side of the join and matching rows for the table on the right side of join (The rows for which there is no matching row on right side, the result-set will contain null)

E.g., `SELECT Student.NAME, StudentCourse.COURSE_ID FROM Student LEFT JOIN StudentCourse ON StudentCourse.ROLL_NO = Student.ROLL_NO;`

Output

Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

LEFT JOIN

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL

Join Operations

Right Outer Join

- Returns all the rows of the table on the right side of the join and matching rows for the table on the left side of join (The rows for which there is no matching row on left side, the result-set will contain null)

E.g., `SELECT Student.NAME, StudentCourse.COURSE_ID FROM Student RIGHT JOIN StudentCourse ON StudentCourse.ROLL_NO = Student.ROLL_NO;`

Output

Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

RIGHT JOIN

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
NULL	4
NULL	5
NULL	4

Join Operations

Full Outer Join

- Combining result of both LEFT JOIN and RIGHT JOIN
- The result-set will contain all the rows from both the tables (The rows for which there is no matching, the result-set will contain NULL values)

E.g., `SELECT Student.NAME, StudentCourse.COURSE_ID FROM Student FULL JOIN StudentCourse ON StudentCourse.ROLL_NO = Student.ROLL_NO;`

Output

Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

FULL JOIN

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL
NULL	9
NULL	10
NULL	11

Writing Optimized Queries

- Given a query, there are generally a variety of methods for computing the answer
- A query could be expressed in several different ways
- Each SQL query can itself be translated into a relational-algebra expression in one of several ways
- Furthermore, the relational-algebra representation of a query specifies only partially how to evaluate a query; there are usually several ways to evaluate relational-algebra expressions
- As an illustration, consider the query:

select salary

from instructor

where salary < 75000;

Writing Optimized Queries

- This query can be translated into either of the following relational-algebra expressions:
 - $\sigma_{salary < 75000} (\Pi_{salary} (instructor))$
 - $\Pi_{salary} (\sigma_{salary < 75000} (instructor))$
- Further, we can execute each relational-algebra operation by one of several different algorithms
- For example, to implement the preceding selection, every tuple in instructor can be searched to find tuples with salary less than 75000
- If a B+-tree index is available on the attribute salary, we can use the index instead to locate the tuples

Writing Optimized Queries

- To specify fully how to evaluate a query, we need not only to provide the relational-algebra expression, but also to annotate it with instructions specifying how to evaluate each operation
- The different evaluation plans for a given query can have different costs
- It is not expected users to write their queries in a way that suggests the most efficient evaluation plan
- Rather, it is the responsibility of the system to construct a query-evaluation plan that minimizes the cost of query evaluation; this task is called query optimization